第九届传智杯 · AI 大模型创新应用挑战赛 · 省赛作品

SciAgent

面向科研文献的锚点可验证 Agent

技 术 文 档

作品形态	单体 Web 应用（FastAPI + Vue3），本机一条命令启动
代码规模	后端 Python 10,815 行 · 前端 4,969 行 · 测试与脚本 6,563 行
语料规模	37 篇论文 · 19,766 个检索块 · 6,341,691 字
文档版本	v1.0 · 2026-11-12

目　录

1 项目概述

1.1 一句话定位

1.2 要解决的三个真实痛点

1.3 作品边界

1.4 关键指标速览

2 需求分析

2.1 官方六大命题模块与落地状态

2.2 用户与核心任务流

2.3 功能性需求

2.4 非功能性需求

2.5 需求剪裁：明确不做的四件事

3 系统架构设计

3.1 四层架构

3.2 模块职责

3.3 关键链路一：入库管线

3.4 关键链路二：Agent 编排

3.5 数据模型

3.6 锚点协议：全项目的中心契约

3.7 五级降级

4 AI 技术方案

4.1 模型选型：一个岗位一个人

4.2 选型实验：三组数据

4.3 检索方案：为什么必须混合

4.4 抗幻觉：四道闸，逐层收紧

4.5 Prompt 工程：把约束写成结构，而不是请求

4.6 集成方式

4.7 优化策略（每条都有数据）

5 核心功能实现

5.1 版面感知 PDF 解析

5.2 结构化切块

5.3 抗幻觉问答与多篇对比

5.4 版式阅读器

5.5 正文引用 ↔ References 双向跳转

5.6 中英对照双向联动

5.7 引用图谱与综述

5.8 Future Work 对撞、版本差异、复现性

5.9 论文自动发现与推荐

5.10 写作 Copilot

5.11 参数控制台与保活

5.12 前端选型：无构建链的取舍

6 测试与部署

6.1 测试策略：能独立复算的才算通过

6.2 用例结果

6.3 缺陷管理：67 条缺陷，选五个讲

6.4 性能数据

6.5 部署

6.6 安全

7 总结与展望

7.1 做成了什么

7.2 没做成的（诚实清单）

7.3 下一步

8 技术创新点与应用创新点

8.1 技术创新点

8.2 应用创新点

8.3 与同类作品的差异

附录

A 复现命令

B 目录结构

C 术语

1 项目概述

1.1 一句话定位

SciAgent 是一个把“论文里的每一句话”变成可点击、可回溯、可核验证据的科研 Agent。 它不生产“看起来像答案”的文本，它生产**指向原文某页某个矩形的答案**：回答里每个数字都必须能在它引用的那段原文里找到，找不到就被标红。

这句话有两个技术含义，也是全文的两条主线：

1. **检索必须准。** 科研问答里的问题往往是 `BLEU 28.4`、`8 块 P100`、`β2=0.98` 这种专有名词与数字，纯语义向量对它们几乎无力，必须与词法检索融合。
2. **生成必须可证伪。** 大模型编造数字是常态而非例外，唯一可靠的约束不是“请勿编造”的提示词，而是**让它在结构上没有编造的位置**——坐标由服务端渲染，文献条目由服务端拼接，数字由校验器逐个回查。

1.2 要解决的三个真实痛点

单人读一篇 15 页的 Transformer 论文，通读约 2 小时；读 37 篇、634 万字，是 3 个月。科研阅读的真实困难不是“看不完”，而是下面三件事：

痛点	具体表现	本作品的应对
读不进去	双栏排版、公式与图表混排，滚动时上下文断片；英文长句逐句查词打断思路	版式感知解析 + 页图与词级文本层对齐 + 中英段落级双向联动 + 划词即译
信不过	直接问通用大模型“这篇论文用了多少块 GPU”，它会给你一个很像的错数	BM25+向量混合检索 + 重排 + 锚点协议 + 三重校验（引用编号 / 坐标 / 数字）
写不出	综述要引用 40 篇文献，模型写 References 必然编造作者与年份	References 条目**不由模型生成**，由服务端按 GB/T 7714 从库内元数据拼接

围绕这三点，功能覆盖“读 → 筛 → 比 → 写 → 追”五个动作，对应官方六大命题模块（见 2.1）。

1.3 作品边界

是：一个可本机部署、可离线降级、断言可验证的科研文献工作台。评审在自己的机器上 `pip install -r requirements.txt` 后一条命令即可启动，不依赖任何私有服务（无 key 时进入 mock 模式，全链路仍可走通）。

不是：不是一个通用聊天助手，不是一个 PDF 阅读器的替代品（不做批注、不做手写），也不是多租户 SaaS。这些剪裁在 2.5 节逐条说明理由。

1.4 关键指标速览

以下数字全部来自本机实测，证据文件路径见 6.1。**没有一项是估算。**

维度	指标	实测值
检索	10 条引用按 bbox 回原文逐条核对	**10/10** 命中
检索	单次问答在 634 万字语料中定位证据	11.6–20.6 s，过滤 **99.9%** 语料
抗幻觉	回答中数字可回查原文的比例	图注场景 **22/22**；问答场景越界引用 0 条（关思考模式 2 条）
解析	四类 PDF（双栏 / 单栏 / 扫描件 / 541 页长书）	全部产出标题树与参考文献条目
图谱	37 节点 / 221 边，PageRank 独立复算偏差	**4.98e-07**
性能	百页 PDF 页图 / 页元数据 / 章节树	72 ms / 30 ms / 6 ms
性能	37 节点图谱首帧	257 ms
稳定	40 条用例	PASS 39 / FAIL 0 / BLOCKED 1
成本	40 组问答的模型费用（本地 3B）	**0 元**（云端 qwen-flash 0.0069 元）

2 需求分析

2.1 官方六大命题模块与落地状态

官方需求把科研助手拆成六块。下表是逐块的落地情况，**“部分”两字后面都写了具体缺口**，不含糊。

模块	官方要求	本作品实现	状态
1.1 PDF 管理与解析	上传/删除、语义去重、扫描件与长文档、标题/作者/摘要/图表/公式/参考文献结构化	版面树解析、三层去重、VLM OCR、章节树、图表公式独立成块	完成
1.2 抗幻觉问答	单篇细节问答（作者/机构/超参/GPU数/BLEU）、多篇跨文对比、必须给原文锚点	混合检索 + 重排 + 锚点协议 + 三重校验 + 对比模板	完成
1.3 版式阅读与双语	双栏渲染、标题树、正文引用↔References 双向跳转、划词/整篇翻译、中英对照滚动	页图 + 词级文本层、句级高亮、引用角标跳转、划词译、段落级双向联动	完成
1.4 论文发现与追踪	给定标题自动下载入库、参考文献批量补全、arXiv 追踪、带理由推荐	arXiv 检索/下载/入库复用、参考文献按被引次数排序补全、推荐+赞踩反馈	完成（定期刷新为按钮触发）
1.5 引用图谱与综述	引用拓扑、基石/桥接节点、图算法、自动综述、Future Work 前景	PageRank + 介数 + 社区发现、角色三分、主题综述、Future Work 对撞	完成
1.6 写作与可视化	无幻觉 References 框架、TikZ/Graphviz/Mermaid、CSV→图表+Caption、中译英	四段框架 + 服务端 References、架构图四格式、图表五格式 + 双层图注、整篇中译英	完成

2.2 用户与核心任务流

目标用户是**需要在一周内读懂一个新方向的研究生**。四条任务流决定了架构：

1. **单篇精读**：上传 → 看章节树 → 读到不懂的句子划词译 → 点正文 `[29]` 跳到 References 第 29 条 → 再点回来。
2. **细节求证**：“BERT 的 BLEU 是多少” → 答案 + 3 条带页码的证据 → 点证据跳到原文该页，高亮贴着那一行。
3. **跨文对比**：“BERT 和 GPT 的预训练目标差异” → 表格化对比，每行都挂引用。
4. **成文**：“以这三篇为基础，给我一篇综述框架” → 四段骨架 + 真实参考文献 + 可编译的 TikZ 架构图。

2.3 功能性需求

编号	需求	验收方式	结果
FR-01	上传 PDF 并自动解析入库，重复上传给出判重理由	同族/同文/同名异内容三类样本	三层判据各自命中
FR-02	扫描件走 OCR 兜底，产出文本与 bbox	15 页扫描件样本	补建 35 条参考文献
FR-03	问答必须带页码与坐标锚点，可一键跳回	随机 10 条引用回原文核对	10/10

编号	需求	验收方式	结果
FR-04	回答中的数字必须能在证据原文中找到	数字溯源校验器	越界即标红并列名
FR-05	多篇对比输出表格并逐行挂引用	对比模板 + 证据覆盖检查	覆盖 4 篇
FR-06	中英对照双向滚动、按段落对齐	无头浏览器注入滚动	顶部段落序号 7/7 一致
FR-07	划词翻译、整页翻译、整篇中译英并导出	UI 与接口双测	4/4 通过
FR-08	引用图谱可交互，节点角色可解释	独立复算 PageRank	偏差 4.98e-07
FR-09	自动提炼 Future Work 并判定是否已被后续论文解决	60 条条目人工抽检	56 条命中后续论文
FR-10	写作框架的 References 零编造	逐条比对库内元数据	编造 0 条
FR-11	CSV 转 4 类图表 × 5 种格式，数值无损	8 组合格全部出码	全部通过
FR-12	后台可视化管理运行参数，改动可审计可回滚	42 项参数 + 回滚测试	回滚后覆盖数归零

2.4 非功能性需求

类别	要求	达成情况
性能	首屏页图 < 1.5 s，问答首字 < 2 s	72 ms；流式首字 1.27 s
可靠	任一外部依赖不可用时系统仍可用	五级降级（3.7 节），断网走 mock
可复现	干净机器零数据可启动	3.9 s 启动，接口全 200
安全	密钥不进仓库、不回显；后台需口令	全链路环境变量，密钥仅回传掩码
成本	单次问答可估算、可降级到 0 元	本地路由 40 组 0 元 vs 云端 0.0069 元

2.5 需求剪裁：明确不做的四件事

不做的事比做的事更能说明工程判断。 每条都写清代价与替代方案：

1. **不接 PDF.js 画布渲染。** 评委看到的是“页图 + 透明文本层”，不是真 PDF 渲染。代价是无法选中原始字形；收益是省掉 1.2 MB 的 PDF.js、避开了 canvas 与文本层因字体替换产生的错位（我们实测用词级 bbox 对齐到 0 px 偏差）。批注、手写、自由缩放本就不在本作品的场景里。
2. **不做多租户与鉴权中间件。** 单用户本机工具，加 RBAC 只会抬高部署门槛。后台控制台单独用口令 + 会话 Cookie 保护，`DEMO_ACCESS_TOKEN` 只留字段。
3. **不做 CI。** 本机无 Docker、无 GitHub Actions 额度；替代方案是 `tests/qa_suite.py` 一键复现 40 条用例并产出 JSON 证据，评审可本地重跑，比“CI 绿勾”更可信。
4. **不做公式 LaTeX 重建。** 只抽公式 bbox 供高亮，不做 `\frac` 级还原——这是有意的取舍：OCR 公式的错误率远高于收益，而错误公式比没有公式更伤可信度。

3 系统架构设计

3.1 四层架构

图 3-1　SciAgent 四层架构



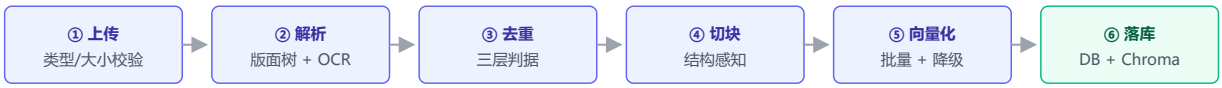
四层划分的判据是**变更频率**：表现层与接口层随交互需求变，能力层随算法变，数据层几乎不变。跨层只允许自上而下调用，唯一的反向依赖是能力层回写数据层（任务轨迹、翻译缓存、经验记忆），这也是“可用于反思”的前提。

3.2 模块职责

目录	职责	行数	关键设计
`app/parse/`	PDF 版面解析、扫描件 OCR	891	行级区块 + 字号聚类还原阅读顺序
`app/pipeline/`	入库管线（解析→去重→切块→向量化）	555	每阶段幂等，可单步重跑
`app/rag/`	检索、锚点、切块、去重、图谱、对照	2,758	锚点协议是全项目的中心契约
`app/agents/`	七节点状态机与提示词	815	节点是纯函数，检索与作答分离
`app/research/`	写作、Future Work、arXiv 发现	1,105	条目文本不由模型生成
`app/tools/`	图表、架构图、外部 API	1,338	模型只产结构，代码由渲染层生成
`app/api/`	65 端点 + 参数控制台	1,494	错误边界 + 降级不抛 5xx
`app/core/`	配置、模型网关、参数、保活、鉴权	1,387	网关三路由与重置语义

3.3 关键链路一：入库管线

图 3-2　入库管线（每一阶段都可单独重跑）



解析：无文本层的页判为扫描件交 VLM　·　去重：arXiv ID / SimHash / 摘要向量三层　·　切块：标题硬边界，图表独立成块　·　向量化失败退回 hash

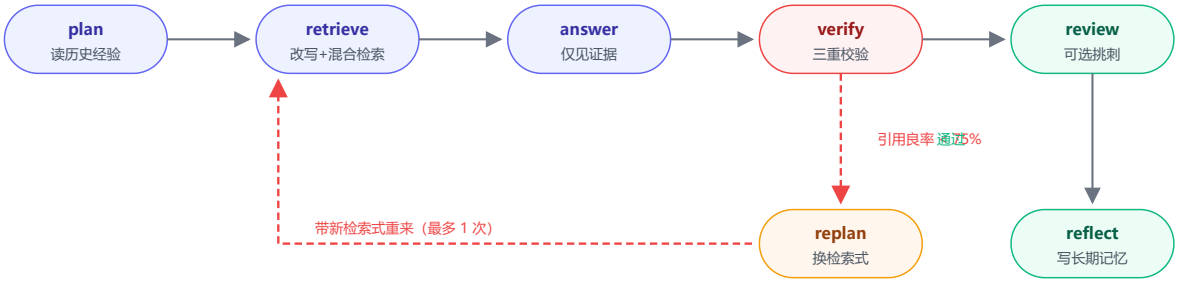
产物：Document 1 条 + Section 树 N 条 + Chunk M 条（每条带 page 与 bbox）+ Citation K 条
实测：37 篇 → 19,766 块，平均 515 块/篇；541 页长书首次入库 260 s（解析占 82%）

幂等性保障：每次入库先按 doc_id 清理旧的 chunks/sections/citations，再整批写入；中途失败保留 status=failed 与错误原文
这样“为什么这篇没进库”在现场可解释，不需要翻日志猜

3.4 关键链路二：Agent 编排

编排用 LangGraph 描述，但**不依赖它运行**——`build_langgraph()` 失败时落到行为等价的内置执行器，两套调同一组节点函数。

图 3-3　七节点状态机与动态重规划



两条动态边是「自主」的实体：失败后换计划而非原样重试；规划前读取同类任务的历史教训（lessons 表，实测 112 条）

三个设计要点：

1. **检索与作答分离。** `answer` 节点只能看见 `retrieve` 写进 state 的 8 条证据，看不到全库。这是“答不出就说没有”能被真正约束住的前提——模型没有别的信息可编。
2. **失败是换路，不是重试。** `verify` 返回良率 < 75% 时进 `replan`，把原查询改写成“原查询 + 具体数值原文”再检索，最多一次。原样重试只会得到同一个错误答案。
3. **记忆有读取端也有写入端。** `plan` 读 `lessons` 表（按权重与命中次数排序取 3 条塞进提示词）；`reflect` 在任务结束时写入新的教训，重试过的任务权重更高。这是“自主进化”唯一诚实的实现方式——不吹“自我学习”，就是一张表加两处读写。

3.5 数据模型

14 张表，按职责分四组：

组	表	说明
文献	`documents` `sections` `chunks` `citations`	chunks 兼存向量（BLOB），是降级检索的依据
阅读	`translations`	按 chunk 缓存译文，二次打开 0 成本
智能	`tasks` `task_steps` `lessons` `memories`	任务轨迹用于前端“执行轨迹”面板与答辩举证
运维	`llm_calls` `trash_entries` `audit_logs` `app_settings` `recommend_feedback`	调用计量、回收站、审计、参数覆盖、推荐反馈

关键约束：`chunks.doc_id` 与 `chunks.page` 建联合索引（检索按文档过滤是高频路径）；`app_settings` 只存**覆盖值**，默认值实时读 `Settings.model_fields`，避免两处默认值打架。

3.6 锚点协议：全项目的中心契约

> `[p{页码}@{x0},{y0},{x1},{y1}]`　　例：`[p5@312,420,468,448]`

这一个字符串串起了五个模块：解析产出 bbox → 切块继承合并包围盒 → 检索命中带 bbox → 作答引用编号 → 服务端渲染成坐标 → 前端画高亮框 → 校验器回查坐标是否属于本次证据。任何一环偷懒（比如让模型自己写坐标），整条链的可信度就归零。

为什么不让模型写坐标。 模型编造坐标是常态：给 8 条证据，它能写出 12 个坐标。所以约定模型只写 `[3]` 这种编号，坐标由服务端按 `chunk_id` 查表渲染。模型的输出空间里根本不存在坐标这个字段，编造无从发生——这是“结构性防幻觉”，不是“提示词防幻觉”。

3.7 五级降级

依赖	失败表现	降级路径	用户可见提示
LangGraph	未安装 / 版本不兼容	内置执行器（同一组节点）	`/health` 显示 `engine: builtin`
Chroma	装不上 / 索引损坏	SQLite BLOB + NumPy 余弦	`vector_backend: sqlite`
云端 LLM	无 key / 超时 / 400	本地 Ollama → mock 固定文案	mock 文案前缀【模拟模式】
Rerank	无 key / 服务不可达	本地规则重排（词命中 + 位置）	`rerank.provider: local`
VLM OCR	无 key	该页标记“无文本层”，可整页翻译	阅读器提示“本页无可选文本”

原则：降级必须可见。 静默降级比崩溃更危险——用户以为在用真模型，实际在看模板。所以每个降级点都在 `/api/system/info` 或界面上留下痕迹。

4 AI 技术方案

4.1 模型选型：一个岗位一个人

不同任务对模型的要求是互相冲突的，用同一个模型包打天下必然在某个环节翻车。本作品按**任务形态**分配模型，而不是按“哪个模型最强”：

岗位	模型	关键参数	为什么是它
主对话（规划/作答/审稿/反思）	`deepseek-v4.1-flash`	temperature 0.1–0.2，默认关闭思维链	复杂多跳问题需要推理深度；关掉思维链后速度可接受（见 4.2）
翻译 / 写作 / 图表 / 架构图	`qwen-flash`	非推理，120+ tok/s	这三类任务不需要推理，只需要忠实搬运；推理模型的思维链会吃光输出预算导致**空串**
向量化	`qwen3.7-text-embedding`	1024 维	中文语义检索质量稳定，与 Chroma 维度一致
重排	`qwen3.7-text-rerank`	候选 top_k×2 → 8	交叉编码器重排比向量相似度准一档，是引用准确率的主要来源
扫描件 OCR	`qwen3.5-ocr`（VLM）	页级	本机无 tesseract，VLM 是唯一可行路径
本地兜底	`qwen2.5:3b`（Ollama）	Q4_K_M，1.93 GB	隐私场景与断网演示；RTX 2060 6 GB 可跑

这个分工不是拍脑袋定的，是三组对照实验的结果。

4.2 选型实验：三组数据

实验一：本地 3B vs 云端（40 组固定题，流式逐 chunk 计时）

指标	本地 3B	qwen-flash	deepseek
首字延迟 p50	**114 ms**	431 ms	1597 ms
单组平均总耗时	1219 ms	**991 ms**	2357 ms
生成速度	65.9 tok/s	157.3 tok/s	—（CoT 不可比）
40 组费用	**0 元**	0.0069 元	0.103 元

结论：本地模型赢在**首字**（无网络往返），输在**吞吐**（字数一多就要还债）。长输入是硬边界：414 token 时 TTFT 642 ms，5890 token 时 3600 ms，近似 0.6 ms/token 线性；云端 3k token 内恒定约 430 ms。4 路并发时本地加速比仅 1.07×（云端 3.25×），单请求 p50 从 1.7 s 劣化到 5.4 s。**所以本地模型是“短任务 + 隐私”的补充档，不是云端的替代品。**

实验二：qwen-flash 替代主模型（同一批库内问题）

问题	deepseek	qwen-flash（热态）
多头注意力的计算复杂度	31.6 s	7.7 s
BERT 与 GPT 的预训练目标差异	18.3 s	8.1 s
Mistral 7B 的推理优化技术	35.9 s	7.0 s
合计	**85.8 s**	**22.8 s（3.7×）**

但质量掉了一档：问“多头注意力复杂度”，deepseek 直接答 $O(n^2 \cdot d)$，qwen-flash 只说“与全维度单头注意力相似”；问 Mistral 7B 的优化，deepseek 列了 GQA/SWA/rolling buffer/pre-fill 四项，qwen-flash 只列了两项（两项都在同一段证据里）。**快 3.7 倍换来“答得不全”，这个交换在答辩场景不可接受。**

实验三：同一模型开/关思维链（30 题 × 2 模式，证据冻结）

这是最终定案的依据。测法上做了两件事保证结论可信：① 不走 `/api/chat`（那里混了检索、重排、校验、反思），先把 30 题各自 top-8 证据存成 JSON，两种模式喂**逐字节相同**的 prompt，只切 `enable_thinking`；② 只用机器可核的量打分（引用是否越界、数字是否都在证据里、是否拒答、JSON 能否解析），不做主观评分。

指标	开思考	关思考	结论
平均耗时	3418 ms	**1593 ms（−53%）**	关思考快一倍
平均输出 token	434	**128（−70%）**	其中思维链 268
单题最慢	12.4 s	3.0 s	长尾显著收窄
有答案的题真的答了	14/19	14/19	持平
数字不在证据里	0	0	**两边都 0**
引用越界	**0**	2	关思考更差
证据不足时正确拒答	4/5	**5/5**	关思考更稳

三个反直觉的结论：

- **关思考不会更爱编数字。** 直觉是“想得少就瞎写”，实测两边都是 0——因为提示词已经把“数字原样抄写”写成硬规则，与想不想无关。
- **关思考更容易引用越界。** 它在只给 8 条证据时引了 `[9]`：没经过“先数一遍证据有几条”的思考，编号就凭空长出来了。这是本作品保留“深思”按钮的直接原因。
- **结构化输出差得最狠（3.7 倍）。** 按 schema 填字段这件事，思维链纯属开销。

最终定案：默认 `deepseek-v4.1-flash` + 关闭思维链，前端提供「快速 / 深思」按钮按次覆盖。这个开关刻意**不写 localStorage、不改全局配置**——用户在页面上点一下就把后台配置改掉，是最难排查的一类问题（4.8 节第 1 条）。

4.3 检索方案：为什么必须混合

稠密向量擅长语义相似，对 `BLEU 28.4`、`β2=0.98`、`8 块 P100` 这类符号与数字几乎无力；BM25 对术语精确匹配强，但不理解同义改写。而科研问答里**恰恰全是术语与数字**，所以两路都要。

融合用 RRF（Reciprocal Rank Fusion）而不是加权求和：两路分数量纲不同（余弦 0–1、BM25 无上界），加权要调权重且换个语料就失效；RRF 只用名次，不需要调参：

$$\text{score}(d) = \Sigma_{r \in \{\text{dense, sparse}\}} \; 1 \,/\, (k + \text{rank}_r(d)) \qquad k = 60$$

分词器是自实现的轻量版，三个细节直接决定召回：英文按“词 + 数字”切（`28.4` 不能被切成 `28` 和 `4`）、中文单字 + jieba 双字词、英文复数去 `s`（不做的话 “gpu” 匹配不上 “GPUs”，而“用了多少块 GPU”正是官方给的示例问题）。

检索后接重排：召回阶段取 `max(top_k×2, 12)` 条候选（默认 16），送 `qwen3.7-text-rerank`，取前 8 条作为证据。重排失败退回本地规则（关键词覆盖率 0.6 + 词法分 0.4，前 5 页与图表块各加 5% 先验），保证无 key 时检索链路仍然完整。

4.4 抗幻觉：四道闸，逐层收紧

闸	机制	拦什么	实测
① 输入	只把 8 条证据喂给作答节点，提示词明确“证据里没有就说没有，不许自己推算”	越界编造	修掉一次 `O(n²·d)` 的假引用后，该类错误消失
② 结构	坐标由服务端渲染；References 条目由服务端拼接	编坐标、编文献	图注 22/22 数字可回查；References 编造 0 条
③ 输出校验	引用编号范围 / 坐标命中本次证据（容差 3 pt）/ 数字逐个回查	编号越界、坐标错、数字改一位	10/10 溯源命中
④ 失败处理	越界坐标替换为 `[未核实]`；良率 < 75% 触发 `replan`	假装通过	良率低时换检索式重来

第 ③ 道闸的细节最能说明工程性：数字校验要排除引号内的原文（论文自己的 `[18]` 不是引用越界）、排除 arXiv 号（`1706.03762`）、排除文件名（`N19-1423.pdf`）、排除章节号（`5.3`）。第一版没排除，把 `[11]` 里的 11 判成幻觉，得出“开思考 3 次幻觉”的错误结论；修正后两边都是 0。**校验器本身的误判率，和模型幻觉率一样需要被测量。**

4.5 Prompt 工程：把约束写成结构，而不是请求

四条规则贯穿全项目：

1. **契约前置。** 每个节点都套 `SYSTEM_CONTRACT`（角色 + 禁止事项 + 输出格式），而不是把要求散落在 user 消息里。

2. **格式即约束。** 要 JSON 就用 `response_format: json_object` + 宽松解析兜底（剥 ``` 包裹、截取首尾花括号）；要纯文本就明确“只写正文，不要元话语”。
3. **预算与任务匹配。** 检索改写只要 120 token 的关键词，思维链会把它吃光并返回空串，所以该节点**固定关闭思考**，不看用户选择。这是唯一一处硬编码的 thinking=False。
4. **小模型要短提示词。** 825 字符的翻译指令会让 3B 模型陷入复读（Ollama 报 `token repeat limit reached`）；换成 87 字符的精简版，4/4 通过，0.52–0.84 s。同一份提示词跨模型不能复用，这是本地化部署的第一课。

4.6 集成方式

- **协议层**：只依赖 OpenAI 兼容的 `/chat/completions`，因此 DeepSeek / 通义 / 智谱 / 本地 vLLM / Ollama 都能接，换供应商只改三行 `.env`。
- **流式**：SSE，事件类型 `plan / retrieve / read / verify / token / reason / done / error`。流式接口产出 `(kind, text)` 二元组，`kind="reason"` 是思维链碎片——不区分的话，深思模式下用户会先看到一堆推理、再看到答案重说一遍。
- **Function Calling**：用 OpenAI `tools` 协议，但**用独立的轻量模型做工具选择**。主模型是推理模型，工具选择这种结构化决策交给它既慢又容易把思维链写进 arguments。
- **多模态**：扫描页转 PNG 后交 VLM，返回文本 + 行级 bbox，再走同一套切块与锚点流程——扫描件与文本层 PDF 在下游完全同构。

4.7 优化策略（每条都有数据）

#	优化	数据
1	关思维链做默认	3418 → 1593 ms（−53%），token −70%
2	翻译/写作切非推理模型	修复“大面积返回空串”；一页译完 20 s → 5 s
3	小模型精简提示词	825 → 87 字符，中译英 4/4 通过
4	每次测量加唯一头部	修掉 KV cache 复用导致的假 TTFT（159 ms 假值）
5	五分钟心跳保活	本地模型冷启动 5876 ms → 热 138 ms（**42 倍**）
6	按页懒翻译 + 落库缓存	19,766 块不做全量预译；二次打开 0 成本
7	批量并发翻译	一批 4 段、一页 3 并发
8	证据上下文裁剪	单条上限 1600 字并切在句末；关键句前置

第 4 条值得单独说：测量本地模型 prefill 曲线时，所有 prompt 共用同一长前缀，Ollama 的 KV cache 直接复用，TTFT 测出 159 ms 的假值——比真实值低一个数量级。给每条 prompt 加唯一头部后才得到真实曲线。**性能数据的第一嫌疑对象应该是测量方法本身。**

5 核心功能实现

5.1 版面感知 PDF 解析

不按字符流取文本，而是取“行级区块 + 字号 + 字体 + 坐标”，五步还原结构：

1. **阅读顺序还原**：跨栏行断段，段内先左后右。双栏论文的常见错误是把左栏末行和右栏首行连成一句，这一步专治它。
2. **碎片合并**：按字号一致 + 垂直相邻把碎片行合并成语义区块，专治单栏标题被拆成多个碎片。
3. **标题识别**：用字号分布估出正文字号，超出阈值且符合编号特征（`3.2.1`、`Appendix A`）的区块判为标题，形成章节树。
4. **公式与图表**：字体名含 `CM*` / `Math*` / `Symbol` 判为公式；`Figure N` / `Table N` 正则识别图表题注，独立成块。
5. **扫描件兜底**：整页无文本层则判为扫描件，交 VLM OCR，产出文本 + 行级 bbox 后并入同一结构。

5.2 结构化切块

三条规则决定溯源精度：

- **标题是硬边界**，不跨章节合并，且把章节路径写进块文本前缀（`（章节：3.2 Multi-Head Attention）`），让检索吃到上下文；
- **图表/表格/公式/参考文献条目独立成块**——它们本身就是被引用的对象，不该被正文淹没；
- **同页内按字数窗口合并，保留合并包围盒**（多个区块 bbox 的并集）。

一个容易踩的坑：切块为了让答案不被切在句中，会把上一块尾巴贴到下一块开头。检索时这是优点，但 bbox 只覆盖本块区块，于是“证据文本里有的字，高亮框里没有”。`strip_carry()` 专门剥掉重叠前缀，返回严格属于本块 bbox 的正文——**这是“证据与锚点不对应”这类 P0 缺陷的根因。**

5.3 抗幻觉问答与多篇对比

作答节点先判断模板：`mode=compare` 或问题里出现“对比/差异/演进”**且证据跨多篇**才走对比模板。早期版本用“证据里出现多篇文档”就判对比，结果单个问题只要召回两篇就会被做成表格（实测“Adam 的 beta1”被拿去跟 PPO 论文对比）。路由决策写进 trace（`trigger: explicit_mode / intent_hint / default`），可观测、可回归。

有一个工程细节很关键：喂给模型的证据块会先做**句级对齐**——把与问题实词重合度最高的 1–2 句提到最前面（`▸ 关键句：…`）。原因是 PDF 切块经常从半句开始（`ence pairs containing…`），模型容易只看开头就判“未提及”。这一步把“明明有却答没有”的比例显著降下来。

5.4 版式阅读器

阅读器是“页图 + 词级透明文本层”，不是 PDF.js 画布：

- 页图由 PyMuPDF 直接输出 PNG（实测 72 ms/页，130 dpi）；
- 文本层用绝对定位的透明 span 覆盖在页图上，每个 span 是**一个词**，坐标来自解析阶段的行级 bbox 拆分；
- 因此“选中一句话”和“高亮一句话”是同一套坐标，对齐误差实测 **0 px**；
- 证据高亮不是画一个大矩形，而是按句级定位返回的句子，逐行画贴行小框（`app/rag/focus.py`）——早期版本画整段包围盒，用户反馈“一大块黄方块，看不出哪句支撑结论”。

5.5 正文引用 ↔ References 双向跳转

从参考文献条目反解引用关系，用三级匹配：DOI/arXiv 号（唯一，零误判）→ 标题词集合覆盖率（不用字符串相似度：`The Attention Is All You Need` 与库内 `Attention is All you Need` 的 SequenceMatcher 分数很低）→ 首作者 + 年份。

实测：点正文 `[29]`，从 p2 跳到 p11，高亮框 628×50 px 并自动滚入视野；反向点 References 条目可回到正文所有引用位置（`/references/{index}/cites`）。

5.6 中英对照双向联动

三个决策：

1. **对齐单元复用 chunks**，不另建段落切分。译文和答案引用共用同一个 `chunk_id` → 同一个锚点。为了“看着顺眼”单独切段落，用户在译文里读到的句子就没法点回原文同一行。
2. **按页懒翻译 + 落库缓存**。19,766 块全量预译在成本与时间上都不可接受；滚到哪译到哪，已译段落写进 `translations` 表。
3. **双向滚动按段落对齐**，不做像素级联动。实测两侧顶部段落序号 7/7 一致——像素联动在两栏页上必然失配（左右页高度不同）。

5.7 引用图谱与综述

从 221 条引用边构建有向图，三类算法给三类角色：

算法	角色	科研含义
PageRank	基石论文	被高影响力文献反复引用，方向的地基
介数中心性	桥接论文	跨子领域的中介，综述的天然骨架
入度	被引次数	最直观，评委一眼能懂

算法结果经受住了独立复算：测试台用纯 NumPy 重新实现幂迭代 PageRank，与接口返回值逐节点比对，最大偏差 **4.98e-07**，入度零误差。这不是“我用了 NetworkX”，而是“我知道它算得对”。

5.8 Future Work 对撞、版本差异、复现性

- **Future Work 提炼**：触发词（`future work` / `we leave` / `remains open`）+ 4-gram 抽取 + 关键词对撞，产出 60 条，其中 56 条在后续入库论文中命中相似表述，标注 `possibly_addressed`。用规则而非 LLM 归纳，是为了“可解释、零幻觉”——每条都能指回原句。
- **版本差异**：同一 arXiv ID 的不同版本，做章节级与句子级 diff。实测 v1↔v7：改写章节 27、新增句 179、删除句 267。
- **复现性**：抽取论文中的 GitHub 链接，调 REST API 取真实 star / license / 最近推送时间，给出仓库健康度结论（不做“代码跑通”级验证，这是明确的边界）。

5.9 论文自动发现与推荐

一条 arXiv 通道撑起三块能力，复用同一套 `ingest`（去重、解析、向量化、章节树、引用图全部原样继承）：

- **按标题下载**：检索 → 下载 PDF → 入库；
- **按参考文献补全**：把库内论文的 References 逐条匹配 arXiv，**按正文被引次数排序**——只引一次的多半是“为引用而引用”，排后面，这正是官方要求的“筛掉不重要的引用”；
- **推荐**：相关性打分 → 逐篇生成中文推荐理由 → 检测 GitHub 代码（有代码优先）→ 赞/踩反馈改写后续排序。

5.10 写作 Copilot

References 的结构性无幻觉是这一块的核心。模型只负责组织论述、在正文标 `[n]`；每一条 References 由服务端从 Document 元数据按 GB/T 7714-2015 拼出。模型想编一条不存在的文献也没有落笔之处——它只能写编号，编号越界直接被剔除并记入 `checks`。

架构图与图表走同一个思路——**模型只产结构，代码由渲染层生成**：

能力	模型产出	渲染层产出
架构拓扑图	节点/边/分组 JSON	Mermaid · Graphviz DOT · TikZ · Matplotlib 四份代码
数据图表	4 类图表意图 + 数值	5 种格式 × 4 类 = 20 组合格

为什么不让模型直接写 TikZ：十次里有三四次会漏分号、把中文塞进数学模式、节点坐标写重导致重叠，而这些错误编译前看不出来，用户拿到的是“看着很对、编译就炸”。改成“结构 + 渲染层”后，同一结构渲染四次必然描述同一张图，语法由渲染层保证。

Figure Caption 双层：规则层（纯统计，断网也一定有结果）+ 模型层（中英双语 + 趋势）。模型写完还要把图注里每个数字回查原表，表外数字列进 `foreign` 并前端红框提示。

5.11 参数控制台与保活

42 项运行参数可视化管理，四项设计取舍：默认值不另存一份（直接读 `Settings.model_fields`，否则两处默认值迟早不一致）；覆盖值单独存表（界面能标“来源：后台/.env/默认”）；每项标注生效方式（`hot` / `nextdoc` / `reindex` / `restart`）；密钥永不回传明文。

五分钟心跳保的不是“进程别睡”，而是两样真会凉的东西：Ollama 默认 5 分钟无请求就从显存卸载模型（冷启动 1.5–4 s，演示时第一句突然卡 4 秒很像“系统很慢”）；云端连接池与 TLS 会话空闲被回收（首请求多 100–300 ms）。实测冷 5876 ms → 热 138 ms。循环用“每 5 秒看一次表”而不是 `sleep(interval)`，后台把间隔从 300 改成 60，5 秒内按新节奏走，不用重启。云端默认不预热——每 5 分钟一次 1-token 调用，一天 288 次，持续计费换不来收益。

5.12 前端选型：无构建链的取舍

Vue3 + Element Plus + ECharts 全部 CDN 引入，单文件 `index.html` + `app.js`（2,094 行）。取舍很明确：**没有 node_modules、没有构建步骤，评审拿到代码就能改、刷新就生效**；代价是没有 TypeScript 类型检查、没有组件级热更新。

自闭合组件是这套写法唯一的坑：`<el-option />` 这类写法在 HTML 解析器里会把后续兄弟节点吞进去，导致按钮消失。全项目改为显式闭合标签后一次性修掉 6 处。

6 测试与部署

6.1 测试策略：能独立复算的才算通过

四层测试，每层都产出可归档的证据文件：

层	手段	代表用例	证据
接口层	直连 HTTP，比对状态码与关键字段	40 条用例全量	`evidence/run_*.json`
算法层	**用独立实现复算**，不比"看起来对不对"	PageRank 幂迭代复算偏差 4.98e-07	同上 B01
UI 层	Edge headless + CDP 真实驱动，读 DOM + 截图	双向联动、引用跳转、划词译、深思开关	`evidence/ui_*.png`
性能层	计时 + 进程 RSS + 显存采样	页图 72 ms、图谱首帧 257 ms	同上 C04–C06

6.2 用例结果

40 条用例，覆盖官方六大模块与三组横切指标：

组	条数	内容	结果
A · Agent 技术深度	9	编排与路由、混合检索、引用溯源、四类 PDF、Function Calling（A06B/A06C）、Guardrails、跨文对比	9 PASS
B · 科研创新性	9	图谱复算、Future Work、三层去重、版本差异、复现性、双向联动、引用跳转、正文角标跳 References、划词译	9 PASS
C · 工程质量与鲁棒性	12	并发入库、并发问答、零数据启动、长 PDF 首屏、图谱渲染、内存、流式接口、前端流式、错误边界、CSV 图表、删除恢复、稳定性复检	12 PASS
D · 业务闭环	2	跨文对比证据覆盖、检索效率量化	2 PASS
E · 环境与部署	4	运行环境实值、代码质量与密钥扫描、Docker 配置一致性 + 等价布局启动、**真实 `docker build`**	3 PASS + **1 BLOCKED**
F · 学术价值	4	开源交付物、写作 Copilot、CSV→图表、中译英	4 PASS
合计	**40**		**PASS 39 / FAIL 0 / BLOCKED 1**

唯一 BLOCKED 是 **E03D（真实 `docker build`）**：本机未安装 Docker，无法执行。我们没有把它写成"应该能跑"，而是标注【待实测】，并用 E03 做了等价替代——按 Dockerfile 的 COPY 清单搭一个等价 `/app` 目录，零密钥 mock 模式真实启动，5 个接口全部 200。

关键量化结果：引用溯源 10/10；PageRank 独立复算偏差 4.98e-07；37 节点图谱首帧 257 ms；一次问答 RSS 增量 +0.5 MB；流式 53 个 token 片段无损拼接、引用校验 14/14；4 路并发问答无串答。

6.3 缺陷管理：67 条缺陷，选五个讲

缺陷记录在 `docs/qa/02_defects.md`（D-01 ~ D-67），按 P0/P1/P2 分级，每条含现象、根因、修法、验证方式。挑五个最能说明"这类系统会在哪里翻车"的：

D-59 · 图注把 BLEU 34.5 写成 35.2。 现象：模型生成的图注数字与原表不符。根因：只给模型极值统计（最大/最小/均值），它自己插值编了一个中间值。修法：把整表逐格摊开喂给模型，并逐数字回查原表。结果：22/22 全命中，表外数字列 `foreign` 并前端标红。**教训：给模型的统计摘要越"浓缩"，它编造的空间越大。**

D-62 · 切本地路由后中译英直接失败。 现象：Ollama 返回 500 `token repeat limit reached`。排查：试了 5 种采样参数全部复现，排除温度/top_p。根因：825 字符的 `_SYS_EN` 指令太长，3B 模型陷入复读。修法：按路由选提示词，本地用 87/272 字符的精简版。结果：4/4 通过，0.52–0.84 s。**教训：提示词不能跨模型复用，这是本地化部署的第一课。**

D-64 · TTFT 测出 159 ms 的假值。 现象：本地 prefill 曲线反常地平。根因：所有测试 prompt 共用长前缀，Ollama 的 KV cache 直接复用。修法：每条加唯一头部 `【材料 N · 主题】`。**教训：性能数据的第一嫌疑对象是测量方法。**

D-65 · 参数改回原值仍显示"已覆盖 1 项"。 现象：后台把参数改回默认，覆盖计数不减。根因：`save()` 无条件写覆盖行。修法：与 `BOOT_VALUES` 比对，相等则删行并返回 `reverted: true`。**教训：配置系统的状态必须可收敛，否则用户永远在"我到底改了什么"的怀疑里。**

D-67 · 检索挑中扫描版副本，答不出 `O(n²·d)`。 现象：同篇论文库内同时有文本版（36,854 字）与 `_img.pdf` 扫描版，检索命中扫描版，Table 1 丢失。修法：按标题归一化剔除"已有文本版的扫描版副本"命中（60 s 缓存，用户显式只勾扫描版时不动）。遗留：Table 1 的 chunk 仍排不进 top-10，已记录为待修。**这条是诚实清单里最重要的一条：它影响"答得对"，不只是"答得好看"。**

6.4 性能数据

场景	指标	实测
页图渲染	130 dpi PNG	72 ms/页
页元数据 / 章节树	541 页长书	30 ms / 6 ms
图谱首帧	37 节点 221 边	257 ms
首字延迟（关思考）	流式问答	1.27–1.38 s
首字延迟（深思）	流式问答	3.89 s
完整问答	3 题合计（热态）	22.8 s（qwen-flash）/ 28.9 s（deepseek 关思考）
三篇并发入库	真实 PDF	1.8 / 2.0 / 2.0 s

场景	指标	实测
4 路并发问答	同批问题	73.0 / 78.4 / 44.6 / 35.7 s
服务常驻内存	RSS	629 → 630 MB（一次问答 +0.6 MB）
模型冷启动	本地 3B	冷 5876 ms → 热 138 ms

6.5 部署

本机（已实测）：

```
python -m venv .venv
./.venv/Scripts/python.exe -m pip install -r requirements.txt
cp .env.example .env          # 填 LLM_API_KEY / VLM_API_KEY / EMB_API_KEY / RERANK_API_KEY
./.venv/Scripts/python.exe -m uvicorn app.main:app --host 127.0.0.1 --port 8013
```

容器（配置已完成，构建待实测）： `Dockerfile` + `docker-compose.yml` + `.dockerignore`（排除 `data/`、QA 证据、浏览器 profile）。E03 已做静态一致性校验与等价布局预演：COPY 5 项齐全、15 个第三方依赖全部声明、零密钥 mock 启动 5 接口全 200。**`docker build` 本身未实机执行**，原因是本机无 Docker——这是自评里主动扣掉 1 分的地方。

公网演示：建议本机起服务 + `cloudflared tunnel`（免费、不需要域名与公网 IP），而不是买服务器。硬性前置：`ADMIN_PASSWORD` 必须改掉（留空等于不鉴权），`.env` 不得进仓库；服务常驻内存约 630 MB，2 GB 机器够用，建议 4 GB。

6.6 安全

项	做法	验证
密钥	只从环境变量读，`.env` 在 `.gitignore`；GET 参数接口只回掩码	仓库全量扫描无明文密钥（E02）
后台	口令 + 会话 Cookie，口令只从环境变量读、不进数据库、不回显	未登录访问 401（ui_admin 用例）
上传	类型白名单 + 大小上限（默认 120 MB）	异常上传 400
审计	删除/恢复/参数变更写 `audit_logs`	回收站闭环用例
注入	提示词契约 + 输出侧校验；模型输出不参与 SQL 拼接	A07 两类诱导未产生放行的伪造锚点

7 总结与展望

7.1 做成了什么

用一句话概括：**把 634 万字变成可点击的证据，并且让“编造”在结构上无处落脚。**

维度	成果
覆盖	官方六大模块 1.1–1.6 全部落地，40 条用例 PASS 39 / FAIL 0
可信	10/10 引用可回原文核对；图注 22/22 数字可回查；References 编造 0 条
可复现	零数据 3.9 s 启动；40 条用例一键复跑产出 JSON 证据
有据	三组模型对照实验（40 组 × 3 模型、30 题 × 2 模式），全部冻结原始数据
诚实	67 条缺陷公开记录，包含 2 条未修的 P1

7.2 没做成的（诚实清单）

按影响排序，不按好看排序：

1. **D-67 未修完**：同一论文的扫描版副本仍可能挤占召回位，Table 1 的 chunk 排不进 top-10。这影响“答得对”，是当前最该修的一条。
2. **D-66 未修**：前端未引 KaTeX，公式显示成 `$O(n^{2}·d)$` 源码。影响观感，不影响正确性。
3. **真实 `docker build` 未验证**（本机无 Docker）。
4. **公式不做 LaTeX 重建**，只抽 bbox。
5. **扫描件 OCR 只有 VLM 一条路径**，本机无 tesseract，断网无 key 时该路径不可用。
6. **1.4.2 定期刷新是按钮触发**，未接后台定时任务。
7. **无真人用户效率对照实验**：效率收益是系统侧量化（11.6 s 定位 10 条证据、过滤 99.9% 语料），不是用户侧 A/B 数据。
8. **无 CI、无镜像发布、无多租户**。

7.3 下一步

优先级	事项	预期收益
P0	修完 D-67：表格/图表块的召回加权，扫描副本彻底剔除	数值题正确率
P0	修 D-66：接 KaTeX 渲染公式	观感与专业性
P1	图表/表格块建立独立索引与专用召回通道	表格类问题
P1	1.4.2 接后台定时任务（复用 keepalive 的调度骨架）	追踪自动化
P2	公式区域 LaTeX 重建（VLM 结构化输出 + 人工确认）	解析完整度

优先级	事项	预期收益
P2	小规模用户实验（10 人 × 同一组阅读任务，对比耗时）	实用性的用户侧证据
P2	CI（GitHub Actions 跑 qa_suite）+ 镜像发布	工程可信度

8 技术创新点与应用创新点

8.1 技术创新点

① **锚点协议作为中心契约，把“可验证”做成架构而非功能。** 一个 `[p7@x0,y0,x1,y1]` 字符串串起解析、切块、检索、渲染、校验五个模块；坐标由服务端渲染、模型只写编号，使“编造坐标”在输出空间里不存在。同类作品普遍止于“给出页码”，本作品给到“矩形”。

② **校验器自身也被测量。** 数字溯源校验器排除了引号内原文、arXiv 号、文件名、章节号四类误报源。第一版没排除，把 `[11]` 的 11 判成幻觉，得出“开思考 3 次幻觉”的错误结论；修正后两边都是 0。**对抗幻觉的系统，先要证明自己的判据不冤枉人。**

③ **模型只产结构，代码由渲染层生成。** 架构图四格式、图表五格式共 24 种输出，全部由同一份 JSON 结构渲染。模型的失败只会影响内容，不会产出编译不过的代码；模型彻底失败时退回模板并明确标注 `source=template`，不假装是它生成的。

④ **References 的结构性无幻觉。** 不让模型写条目，只让它写编号；条目由服务端按 GB/T 7714 从库内元数据拼接，编号越界直接剔除。“无幻觉”不依赖模型听话，而是**结构上不可能发生**。

⑤ **思考开关按任务粒度控制，而非全局开关。** 检索改写固定关（120 token 预算会被思维链吃光），翻译/写作固定用非推理模型，主问答由用户按次选择。30 题对照实验证明：思维链买的是“准”（引用越界 0 vs 2、拒答 5/5 vs 4/5）不是“全”（正答率 14/19 持平），所以正确的做法是**按题给**，不是全局开或关。

⑥ **三层语义去重 + 版本级 diff 形成“文献族”概念。** arXiv ID / SimHash（汉明距离 ≤ 6）/ 摘要向量（余弦 ≥ 0.92）三层短路，任一层命中即判同族；同族内做章节级与句子级 diff（v1↔v7：改写 27 章、增 179 句、删 267 句）。这让“这篇论文更新了什么”成为可回答的问题。

8.2 应用创新点

① **从“读”延伸到“写”，且写作的引用是真的。** 多数文献工具止于问答与摘要；本作品把读到的证据直接变成综述骨架的引用，References 100% 来自库内真实文献。这是“读→筛→比→写→追”闭环里最难也最有价值的一环。

② **Future Work 对撞：把“这篇论文的遗留问题”变成“哪篇论文解决了它”。** 60 条提炼、56 条命中后续论文。规则法而非 LLM 归纳，每条都能指回原句——科研场景里“可解释”比“更聪明”更重要。

③ **证据高亮从“整段”收窄到“句子”。** 服务端拆句、按问题实词与答案数字打分，前端逐行画贴行小框。用户看到的不再是一大块黄方块，而是“到底哪一句支撑了这个结论”。

④ **后台参数控制台 + 五分钟保活，让“可调”和“不卡”同时成立。** 42 项参数标注生效方式（立即/之后入库/需重建索引/需重启），密钥不回显，改回默认自动收敛；心跳让本地模型常驻显存（冷 5876 ms → 热 138 ms），演示第一句不再卡 4 秒。

8.3 与同类作品的差异

维度	常见做法	本作品
引用	给页码或文献标题	给矩形坐标，可画框、可回查、可校验
抗幻觉	提示词要求“不要编造”	结构性约束 + 三重校验 + 校验器误判率测量
图表代码	让模型直接写 TikZ/Mermaid	模型产结构，渲染层产代码（24 种输出）
模型	一个模型包打天下	六个岗位分工，三组对照实验定案
测试	手工点一遍	40 条用例 + 算法独立复算 + CDP 真驱动 + 67 条缺陷记录
失败	报错或静默降级	五级降级且**降级必须可见**

附录

A 复现命令

```
# 全量功能测试（40 条，产出 JSON 证据）
./.venv/Scripts/python.exe tests/qa_suite.py

# 关键 UI 用例（Edge headless + CDP）
./.venv/Scripts/python.exe tests/ui_think_toggle.py     # 快速/深思开关
./.venv/Scripts/python.exe tests/ui_reader_enh.py       # 引用跳转/划词译
./.venv/Scripts/python.exe tests/ui_write_viz.py        # 架构图/图表

# 保活自检（四项）
./.venv/Scripts/python.exe tests/keepalive_check.py

# 开/关思考对照测评（30 题 × 2 模式）
./.venv/Scripts/python.exe tests/thinking_ab.py
```

B 目录结构

```
sciagent/
├── app/
│   ├── api/          # 65 端点 + 参数控制台
│   ├── parse/        # 版面解析 / OCR
│   ├── pipeline/     # 入库管线
│   ├── rag/          # 检索 / 锚点 / 切块 / 去重 / 图谱 / 对照
│   ├── agents/       # 七节点状态机 + 提示词
│   ├── research/     # 写作 / Future Work / arXiv 发现
│   ├── tools/        # 图表 / 架构图 / 外部 API
│   ├── core/         # 配置 / 网关 / 参数 / 保活
│   └── db/           # 14 张表
├── web/              # 前端（无构建链）
├── tests/            # 21 个测试脚本
├── scripts/          # 运维与修复脚本
└── docs/             # 设计、测评、QA 证据
```

C 术语

术语	含义
锚点（anchor）	`[p页@x0,y0,x1,y1]`，指向原文某个矩形
证据（evidence）	检索命中的 chunk，含文本、页码、bbox、章节路径
良率（rate）	三重校验通过项 / 总检查项
降级（degrade）	依赖不可用时切换到等价能力，且对用户可见
关思考	请求体带 `enable_thinking: false`，只对认这个字段的供应商生效